Ambiguous Measurements

AlgoExpert · Hard · Recursion

The problem

- Measuring cups each measure a [low, high] range.
- Can repeated pours produce a volume within [low, high]?
- Memoised search over remaining (low, high) targets.

Intuition

- Try every cup recursively, shrinking the target range, and memoise the ranges already tried.

Brute force vs optimal

- Try all cup sequences → exponential
- Memoised recursion on (low, high) → O(low · high · cups)
- Clamp at 0

Approach

- 1. If a cup fits inside the range, success
- 2. Otherwise subtract the cup from both bounds and recurse
- 3. Memoise by range

Worked example

	min	max
cup A	200	210
cup B	450	465
cup C	800	850

```
target range [2100, 2300]
```

Can we always land in [2100, 2300]?

Worked example

```
use C: [2100, 2300] → [1300, 1450]

use C again: → [500, 600]

use B: → [50, 135] ... then A: covers ✓
```

Subtract a cup's range and recurse

Worked example

```
remaining [low, high] fits a cup: min ≥ low and max ≤ high → done

memoise (low, high)
```

Memoise on (low, high)

Java solution

```java
public class AmbiguousMeasurements {
    public static boolean ambiguousMeasurements(int[][] cups, int low, int high) {
        return can(cups, low, high, new HashMap<>());
    }

    private static boolean can(int[][] cups, int low, int high, Map<String, Boolean> memo) {
        String key = low + ":" + high;
        if (memo.containsKey(key)) return memo.get(key);
        if (low <= 0 && high <= 0) return false;
        boolean ok = false;
        for (int[] c : cups) {
            if (c[1] == 0) continue; // an empty cup never makes progress (avoids infinite recursion)
            if (c[0] >= low && c[1] <= high) { ok = true; break; }
            if (can(cups, Math.max(0, low - c[0]), Math.max(0, high - c[1]), memo)) { ok = true; break; }
        }
        memo.put(key, ok);
        return ok;
    }
}
```

Complexity

- Time: O(low * high * n)
- Space: O(low * high)

Edge cases & pitfalls

- Clamp low at 0
- Both ≤ 0 → fail
- Zero-capacity cups

Interview tips & follow-ups

- Explain the state choice

Recap

- Try every cup recursively, shrinking the target range, and memoise the ranges already tried.
- Complexity: O(low * high * n)